

Lista de Código: Deploy e Variáveis de Ambiente

Turma: LionsDev

Tópicos: variáveis de ambiente (`process.env`), porta dinâmica (`process.env.PORT`), `.env` com `dotenv` e `.gitignore`, script `start` no `package.json` e preparação da API para produção (Render).

Nesta lista você prepara a API para rodar fora da sua máquina. A tarefa é substituir valores fixos no código por configurações lidas do ambiente, para que a mesma API rode localmente e em produção sem alterações.

O Node não lê o arquivo `.env` sozinho. Para carregar as variáveis dele em `process.env`, instale o pacote com `npm install dotenv` e coloque `import "dotenv/config";` na primeira linha do arquivo principal. Em produção, o Render injeta as variáveis configuradas no painel diretamente em `process.env`.

Parte 0: Exercícios de fixação

Itens curtos em que cada valor fixo no código passa a ser lido do ambiente.

1. Imprima o valor de `process.env.PORT`.
2. Defina a porta com fallback: `const PORT = process.env.PORT || 3000`.
3. Leia a string de conexão: `const uri = process.env.MONGO_URI`.
4. Faça o app escutar em `PORT` (não em um número fixo).
5. Crie um arquivo `.env` com `PORT=3000` e `MONGO_URI=...` e carregue-o com `import "dotenv/config";`.
6. Adicione `.env` ao `.gitignore` (por que ele não pode ser enviado ao Git?).
7. No `package.json`, crie o script `"start": "node index.js"`.
8. Leia o segredo do token: `process.env.JWT_SECRET`.
9. Crie uma rota `GET /health` que responde `{ status: "ok" }`.
10. Imprima no boot: `console.log("Servidor rodando na porta", PORT)`.

Parte 1: Complete o código

1. Porta dinâmica

Complete para a API funcionar tanto local quanto no servidor.

```
// TODO: use a porta do ambiente, com 3000 como padrão
const PORT = /* ... */;

app.listen(PORT, () => {
  console.log(`Servidor no ar na porta ${PORT}`);
});
```

2. Conexão via ambiente

Complete a conexão lendo a URI do ambiente.

```
// TODO: pegue a string de conexão da variável de ambiente
const MONGO_URI = /* ... */;

await mongoose.connect(MONGO_URI);
```

3. Script de start

Complete o `package.json` para o Render saber como iniciar a API.

```
{
  "scripts": {
    "start": "/* TODO */"
  }
}
```

Parte 2: Ache o problema

4. Porta fixa no código

Esta API funciona na máquina do aluno, mas falha no Render. Por quê? Conserte.

```
app.listen(3000, () => console.log("rodando")); // problema em produção
```

5. Segredo no código

Este código expõe credenciais, que ficam visíveis para qualquer pessoa com acesso ao repositório no GitHub. Aponte o risco e diga como corrigir.

```
const MONGO_URI = "mongodb+srv://admin:senha123@cluster0.mongodb.net/loja";  
// BUG grave  
const JWT_SECRET = "meusegredo"; // BUG grave
```

Parte 3: Prever o comportamento

6. Qual valor sai?

Suponha que o `.env` tem `PORT=8080`, não tem `MONGO_URI` nem `DEBUG`, e que foi carregado com `import "dotenv/config";`. Diga o valor final de cada constante. Depois, responda: o que muda em (a) se o `import "dotenv/config";` for esquecido?

```
const PORT = process.env.PORT || 3000; // (a)  
const URI = process.env.MONGO_URI || "local"; // (b)  
const DEBUG = process.env.DEBUG; // (c)
```

Parte 4: Prepare para o deploy

7. Checklist de Produção (Desafio)

Pegue uma API sua dos módulos anteriores e deixe-a pronta para deploy no Render. Entregue o código/config para cada item:

1. `PORT` lida de `process.env.PORT` com fallback.
2. `MONGO_URI` e `JWT_SECRET` lidas de `process.env` (nenhum valor fixo no código).
3. Arquivo `.env` criado, carregado com `dotenv` e listado no `.gitignore`.
4. Script `"start"` no `package.json`.
5. Rota `GET /health` respondendo `{ status: "ok" }`.
6. Um `README` curto explicando quais variáveis de ambiente configurar no painel do Render.

Ao final, descreva em 3 passos como você faria o deploy conectando o repositório do GitHub ao Render.

Dica: o código vai para o Git; os segredos ficam no ambiente. A URI do banco e o segredo do token nunca devem estar escritos no código, e a porta deve vir de `process.env.PORT`. Localmente, os valores vêm do `.env`, carregado pelo `dotenv`; em produção, configure-os no painel do serviço. O `.env` fica fora do Git, sempre listado no `.gitignore`.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: Deploy e Variáveis de Ambiente - Módulo 10